ENGSE206 | Workshop สัปดาห์ที่ 11

Conceptual Architecture v0.3 — ตัวอย่างงานที่ทำเสร็จแล้ว

กรณีศึกษา: Campus Resource Booking System (CRBS)

เอกสารนี้คืออะไร

- ตัวอย่างงาน W11 ที่ทำเสร็จแล้วของผู้สอน ต่อยอดจาก Design Strategy (W10) ของ CRBS — ใช้ดูว่างานที่เสร็จหน้าตาเป็นอย่างไร ก่อนทีมทำ case ของตัวเองในแบบฟอร์มเดียวกัน
- แต่ละหัวข้อมี **มาลองทำไปด้วยกัน** (เราทำอย่างไร) → ผลลัพธ์ (ตาราง/ภาพ) → **ทำไมเขียนแบบนี้** — งานของทีมใส่เฉพาะผลลัพธ์
- CRBS เป็นกรณีสอน ไม่ใช่นโยบายจริงของมหาวิทยาลัย; stack ในเอกสารเป็น stack อ้างอิงของรายวิชา (Proposed)

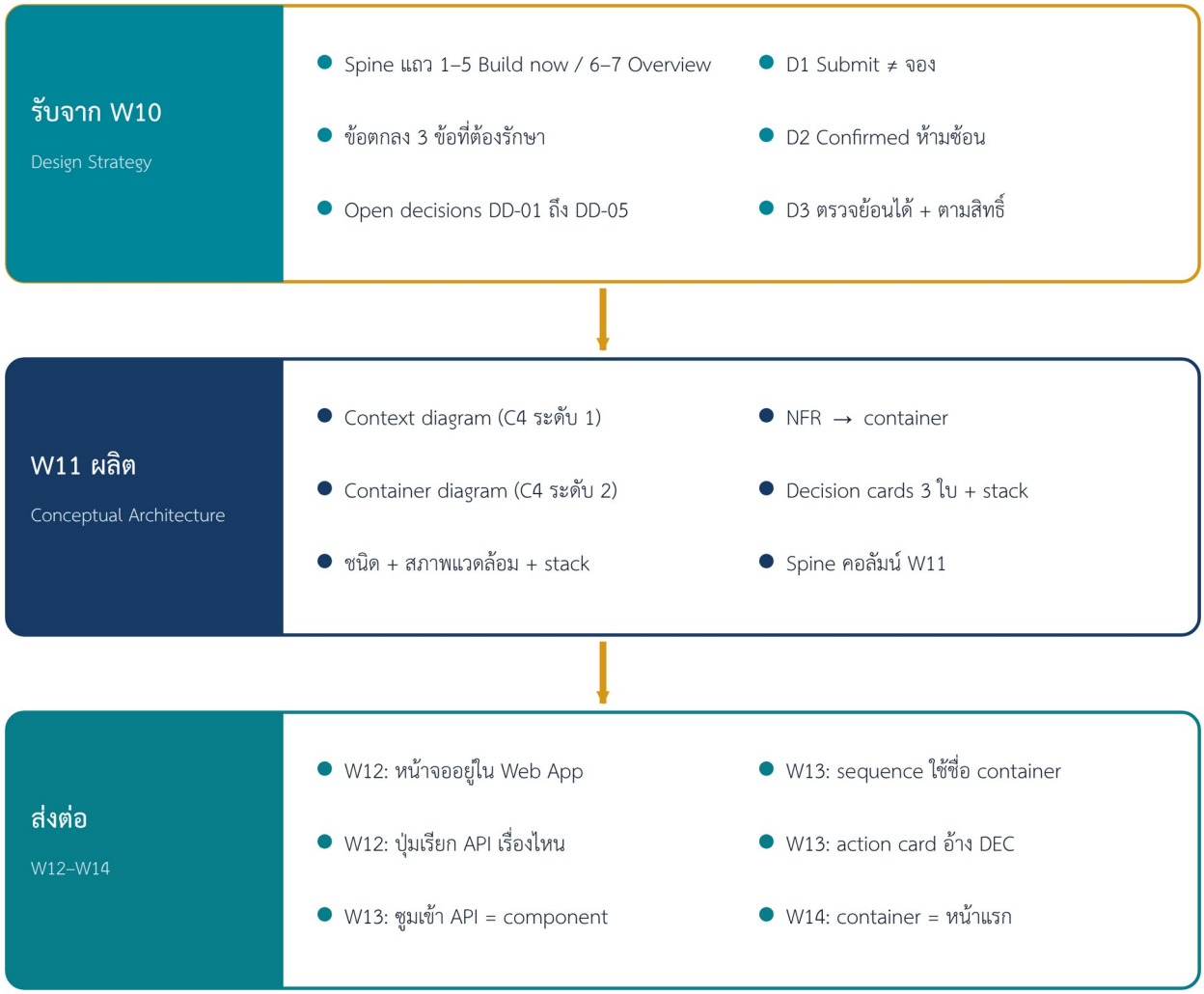
Team	ผู้สอน (ตัวอย่าง)
Case	CRBS — Campus Resource Booking System
เวอร์ชัน/สถานะ	v0.3 — Design Draft
ฐาน SRS	CRBS-SRS-W07-v1.1 — Baseline Candidate (not approved baseline); OI-01–OI-08 ยังเปิด
ข้อมูลเข้า	Design Strategy v0.1 + Design Spine v0.1 (W10)
Stack อ้างอิง	React · Node.js + Express · PostgreSQL — Proposed (ยืนยันกับ Dev: DD-06)
Change request ที่รู้	CR-01-SIM — Pending (ไม่นำมาใช้ในแบบนี้)

1. สิ่งที่รับมาจาก W10

งาน W11 ไม่ได้เริ่มใหม่ — เราหยิบ 4 อย่างจาก Design Strategy (W10) มาเป็นโจทย์ แล้วผลิตงานที่ W12–W13 จะใช้ต่อ (ภาพที่ 1)

สิ่งที่รับ	สาระจาก W10 ของ CRBS	ใช้ใน W11 ตรงไหน
Scope	Build now = ค้นหา · Draft · Submit · ตัดสิน · ดูสถานะ; Overview = Check Out · Return; ยังไม่ทำ = ดึงตารางเรียน · ส่งข้อความภายนอก	ขีดขอบระบบ (หัวข้อ 2) และเลือก container ที่รุ่นแรกต้องมี (หัวข้อ 3)
Design drivers	D1 Submit ≠ จอง · D2 Confirmed ห้ามซ้อน · D3 ตรวจย้อนได้ + ทำตามสิทธิ์	เหตุผลว่ากติกาอยู่ container ไหน (หัวข้อ 4–6)
Design Spine	เส้นทางคำขอหนึ่งใบ 7 ขั้น (ขั้น 1–5 Build now · 6–7 Overview)	เดินทดสอบ container diagram (หัวข้อ 7)

สิ่งที่รับ	สาระจาก W10 ของ CRBS	ใช้ใน W11 ตรงไหน
Open decisions	DD-01 ถึง DD-05	ติดป้าย TBD บนภาพ และเพิ่ม DD-06 (หัวข้อ 8)



หลักคิด: W11 ไม่เริ่มใหม่ — ทุกขั้นของ Spine เดินผ่าน container ได้ และทุก driver มีที่อยู่ฝั่งที่ทีมควบคุม

ภาพที่ 1 W11 ในเส้นทาง: รับอะไรจาก W10 ผลิตอะไร และส่งต่ออะไรให้ W12–W13

ทำไมเขียนแบบนี้

- ถ้าไม่เริ่มจาก W10 ภาพที่วาดจะมาจากความชอบ ไม่ใช่จากงานที่ Dev ต้องสร้างจริง — ตารางนี้บังคับให้ทุกชิ้นมีเหตุผลรองรับ

2. System Context (C4 ระดับ 1)

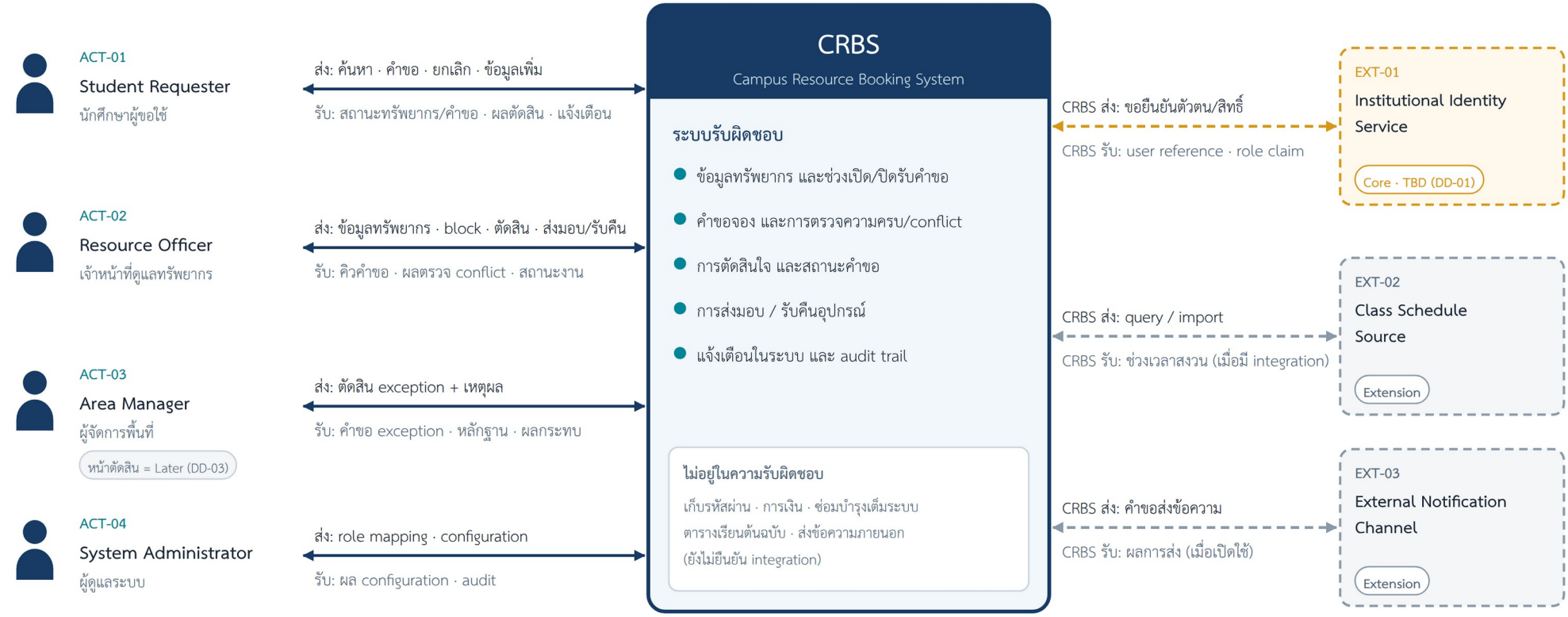
ระบบ 1 กล่อง + ใครมาใช้ + ระบบภายนอก — ตอบคำถามว่า **อะไรอยู่ในระบบเรา อะไรอยู่ข้างนอก** ยังไม่มีกล่องย่อยในภาพนี้ (ภาพที่ 2 อยู่ท้ายหัวข้อนี้)

> **มาลองทำไปด้วยกัน**
>
> 1. วาดระบบของเราเป็น **กล่องเดียว** ก่อน — ยังไม่แบ่งข้างใน
> 2. เปิดตาราง actor ใน SRS หัวข้อ 2 แล้ววาด ACT-01 ถึง ACT-04 รอบกล่อง
> 3. เปิด SRS หัวข้อ 8 (External Interfaces) วาด EXT-01 ถึง EXT-03 และติดป้าย Core / Extension / TBD ตามสถานะใน SRS
> 4. ลากลูกศรทุกเส้นพร้อมป้ายว่า **ส่งอะไร รับอะไร**

ID	ชื่อ	ส่งเข้า CRBS	รับจาก CRBS	สถานะในรุ่นแรก
ACT-01	Student Requester	ค้นหา · Draft/คำขอ · ยกเลิก · ข้อมูลเพิ่ม	สถานะทรัพยากร/คำขอ · ผลตัดสิน · แจ้งเตือน	Core
ACT-02	Resource Officer	ข้อมูลทรัพยากร · block · ตัดสิน · ส่งมอบ/รับคืน	คิวคำขอ · ผลตรวจ conflict · สถานะงาน	Core
ACT-03	Area Manager	ตัดสิน exception + เหตุผล	คำขอ exception · หลักฐาน · ผลกระทบ	อยู่ในภาพรวม; หน้าตัดสิน = Later (DD-03)
ACT-04	System Administrator	role mapping · configuration	ผล configuration · audit	รุ่นแรกใช้ role fixture (DD-01)
EXT-01	Institutional Identity Service	user reference · role claim	คำขอยืนยันตัวตน/สิทธิ์	Core boundary; protocol และ field = TBD (DD-01)
EXT-02	Class Schedule Source	ช่วงเวลาสงวน (เมื่อมี integration)	query / import	Extension — รุ่นแรกใช้ block ที่เจ้าหน้าที่บันทึก
EXT-03	External Notification Channel	ผลการส่ง (เมื่อเปิดใช้)	คำขอส่งข้อความ	Extension — รุ่นแรกใช้ in-app notification

ระบบรับผิดชอบ: ข้อมูลทรัพยากรและช่วงเปิด/ปิดรับคำขอ · คำขอจอง · การตรวจความครบและ conflict · การตัดสินใจ · สถานะ · การส่งมอบ/รับคืน · แจ้งเตือนในระบบ · audit trail

ไม่รับผิดชอบ: การเก็บรหัสผ่าน · การเงิน · การซ่อมบำรุงเต็มระบบ · ตารางเรียนต้นฉบับ · การส่งข้อความภายนอกเมื่อยังไม่ยืนยัน integration



ที่มา: Actor and System Context (ACT-01–04, EXT-01–03) · SRS W07 หัวข้อ 2 และ 8 · ไม่ระบุ protocol หรือเทคโนโลยี

ภาพที่ 2 Context diagram ของ CRBS (C4 ระดับ 1) — ระบบ 1 กล่อง ผู้ใช้ และระบบภายนอก

ทำไมเขียนแบบนี้

- Context diagram ไม่มีกล่องภายใน — ถ้าเริ่มวาดกล่องย่อยตั้งแต่ภาพนี้ จะมองไม่เห็นขอบเขต
- ระบบภายนอกที่ยังไม่ตัดสินต้องติดป้าย TBD หรือ Extension และวาดเส้นประ ไม่วาดเหมือนตัดสินแล้ว
- ลูกศรทุกเส้นมีป้ายว่าส่งอะไร — ลูกศรไม่มีป้ายไม่ได้บอก Dev อะไรเลย

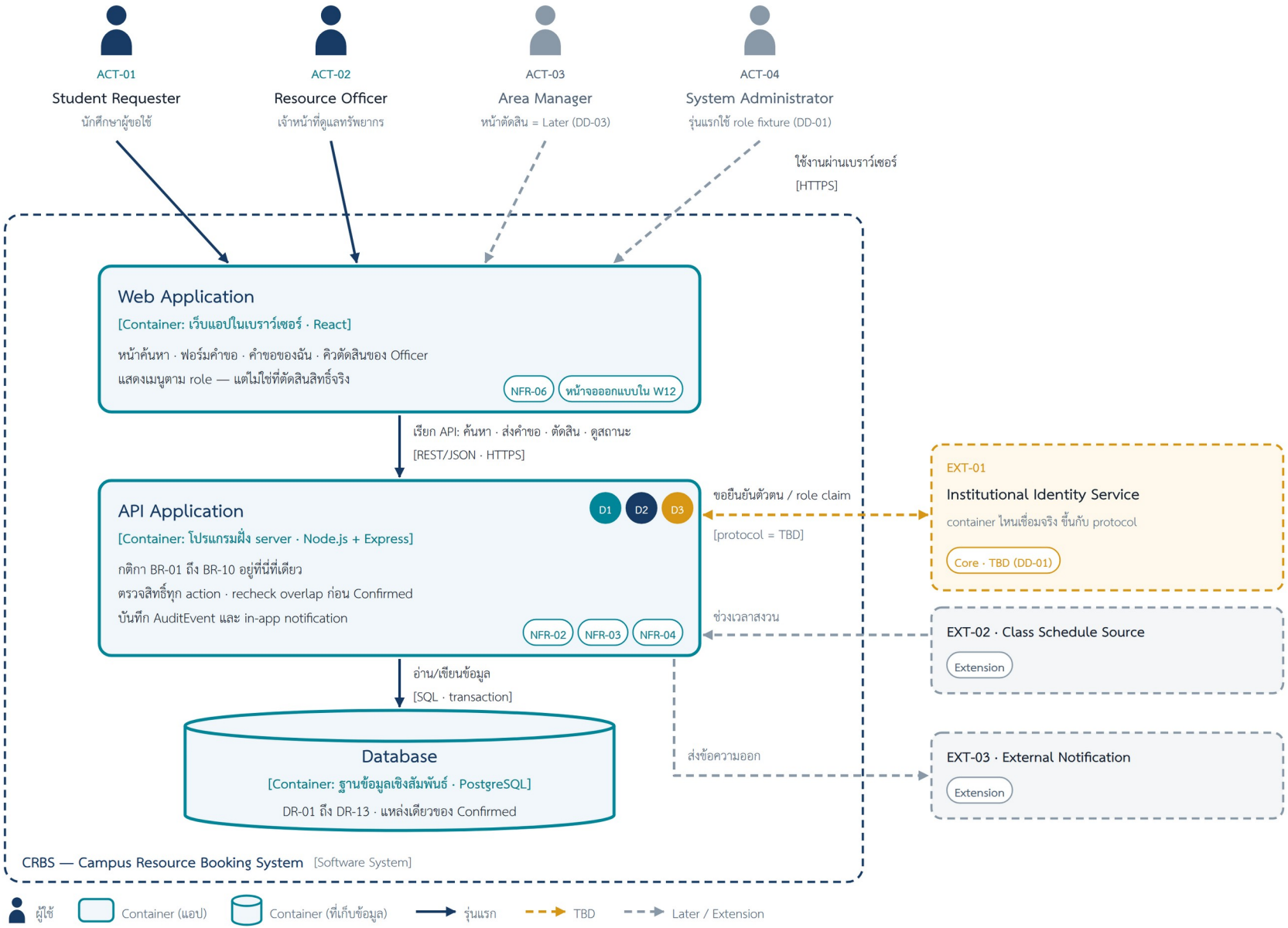
3. Container diagram (C4 ระดับ 2)

ซูมเข้าไปในกล่อง CRBS: ระบบต้องมี **3 ชิ้นที่ต้องรัน** — เว็บแอป · โปรแกรมฝั่ง server · ฐานข้อมูล — แต่ละชิ้นบอกชนิด เทคโนโลยี และหน้าที่ (ภาพที่ 3 อยู่ท้ายหัวข้อนี้)

มาลองทำไปด้วยกัน

1. ถาม **ผู้ใช้ใช้ระบบผ่านอะไร** — ทุก actor ใช้หน้าจอทั่วไปบนเบราว์เซอร์ → Web Application
2. ถาม **กติกา BR-01–BR-10 และการตรวจสิทธิ์ต้องอยู่ที่ไหน** — ต้องอยู่ฝั่งที่ทีมควบคุม → API Application
3. ถาม **ข้อมูล DR-01–DR-13 เก็บที่ไหน** — ต้องเก็บถาวรและมี transaction → Database
4. ใส่ [ชนิด · stack] ใต้ชื่อทุกกล่อง และเขียนป้ายบนลูกศรว่าส่งอะไร ผ่านอะไร — ในตารางบอกด้วยว่าแต่ละชิ้น **รันที่ไหน ใครควบคุม** (ดูหัวข้อ 4)
5. ตรวจว่าระบบภายนอกจากภาพที่ 2 ยังอยู่ครบ และต่อกับ container ที่ถูกชี้น

Container	หน้าที่	ชนิด · stack (Proposed)	รันที่ไหน · ใครควบคุม	ข้อมูลที่เก็บ	คุยกับ (ส่งอะไร)
Web Application	หน้าจอของ Requester และ Officer: ค้นหา · ฟอร์มคำขอ · คำขอของฉัน · คิวตัดสิน; แสดงเมนูตาม role	เว็บแอปในเบราว์เซอร์ · React	เบราว์เซอร์บนเครื่องผู้ใช้ — ผู้ใช้ควบคุม	ไม่เก็บข้อมูลถาวร	API Application — เรียก API ค้นหา · ส่งคำขอ · ตัดสิน · ดูสถานะ [REST/JSON · HTTPS]
API Application	กติกา BR-01–BR-10 · ตรวจสิทธิ์ทุก action · precheck/recheck overlap · เปลี่ยน state · บันทึก AuditEvent และ in-app notification	โปรแกรมฝั่ง server · Node.js + Express	server ของทีม — ทีมควบคุม	ไม่เก็บเอง — อ่าน/เขียนผ่าน Database	Database [SQL · transaction] · EXT-01 [protocol = TBD] · EXT-02/03 (Extension)
Database	เก็บข้อมูลทั้งหมดของระบบ · แหล่งเดียวของ Confirmed	ฐานข้อมูลเชิงสัมพันธ์ · PostgreSQL	server ฐานข้อมูล — ทีมควบคุม · เข้าถึงผ่าน API เท่านั้น	DR-01–DR-13	API Application เท่านั้น



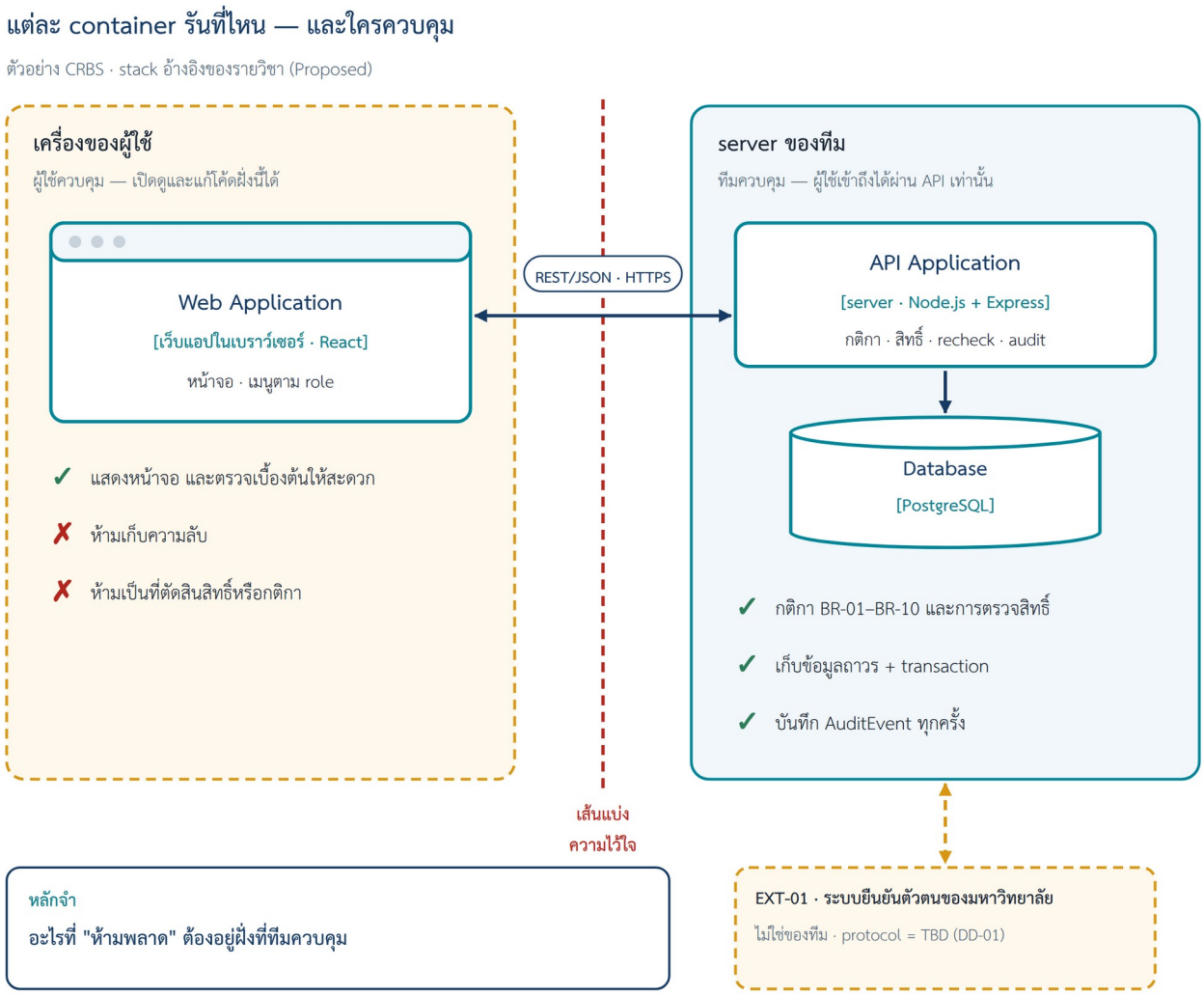
ภาพที่ 3 Container diagram ของ CRBS (C4 ระดับ 2) — 3 ชิ้นที่ต้องรัน ชนิด · stack และเส้นที่บอกว่าส่งอะไร

ทำไมเขียนแบบนี้

- Container คือสิ่งที่ต้อง "รัน" เพื่อให้ระบบทำงาน (แอปหรือที่เก็บข้อมูล) — ไม่ใช่ Docker container และไม่ใช่โฟลเดอร์ของโค้ด
- หน้าเว็บไม่ต่อฐานข้อมูลตรง ทุกอย่างผ่าน API — จึงมีจุดตรวจกติกาและสิทธิ์จุดเดียว
- เริ่มจาก 3 ชิ้นก่อน แล้วเพิ่มเฉพาะเมื่อ requirement บังคับ — SRS ของ CRBS ไม่มี FR/NFR ที่ต้องใช้แอปมือถือหรืองานตามเวลา จึงไม่เพิ่ม

4. ชนิดและสภาพแวดล้อมของ container

ก่อนเลือกเทคโนโลยี เราต้องรู้ว่าแต่ละชนิด **รันที่ไหน และใครควบคุม** — เพราะเป็นตัวตัดสินว่ากติกาและความลับวางที่ไหนได้ (ภาพที่ 4)



ภาพที่ 4 แต่ละ container รันที่ไหน และใครควบคุม — เส้นแบ่งความไว้ใจของ CRBS

เมนูชนิด container — ใช้เลือกชนิดที่ case ต้องการ

ชนิด container	รันที่ไหน / ใครควบคุม	เหมาะกับ	ข้อควรระวัง	ตัวอย่างเทคโนโลยี
เว็บแอปในเบราว์เซอร์	เครื่องของผู้ใช้ — ผู้ใช้ควบคุม	หน้าจอที่ใช้ได้หลายอุปกรณ์ ไม่ต้องติดตั้ง	ผู้ใช้แก้หรือข้ามได้ ห้ามเก็บความลับ ห้ามเป็นที่ตัดสินสิทธิ์หรือกติกา	React · HTML/CSS/JavaScript · Vue
แอปมือถือ	โทรศัพท์ของผู้ใช้	ต้องใช้กล้อง ตำแหน่ง แจ้งเตือนบนเครื่อง หรือใช้ตอนไม่มีเน็ต	ต้องติดตั้งและอัปเดตผ่าน store งานเพิ่มมาก	Flutter · React Native
แอป desktop	คอมพิวเตอร์ของผู้ใช้	ต้องเปิดค้างทั้งวัน หรือใช้อุปกรณ์ในเครื่อง	ต้องติดตั้งทุกเครื่อง	Electron
โปรแกรมฝั่ง server (API)	server ที่ทีมควบคุม	กติกาธุรกิจ ตรวจสิทธิ์ เชื่อมระบบภายนอก บันทึก audit	ต้องออกแบบ API ให้ชัด ต้องเปิดตลอด	Node.js + Express · Python + FastAPI · PHP + Laravel
ฐานข้อมูล	server ฐานข้อมูล — ทีมควบคุม	เก็บข้อมูลถาวร ค้นหา transaction	ให้ API เข้าถึงเท่านั้น ไม่ให้หน้าเว็บต่อตรง	PostgreSQL · MySQL/MariaDB
งานตามเวลา (scheduled job)	server ของทีม	งานที่ต้องเกิดตามเวลาโดยไม่มีคนกด เช่น เตือนก่อนถึงเวลา	เพิ่มเมื่อมี requirement บังคับเท่านั้น	ตัวตั้งเวลาบน server (cron)

CRBS เลือกอะไร

ชนิด	CRBS ใช้ไหม	เหตุผลจาก requirement
เว็บแอปในเบราว์เซอร์	ใช้	ทุก actor ใช้ผ่านหน้าจอทั่วไป · NFR-06 (flow หลักใช้ keyboard ได้)
แอปมือถือ	ไม่ใช้ในรุ่นแรก	SRS ไม่มี requirement ที่ต้องใช้กล้อง ตำแหน่ง หรือใช้งานออฟไลน์
แอป desktop	ไม่ใช้	ไม่มีงานที่ต้องเปิดค้างทั้งวันหรือใช้อุปกรณ์ในเครื่อง
โปรแกรมฝั่ง server (API)	ใช้	กติกา BR-01–BR-10 และ NFR-02/03/04 ต้องอยู่ฝั่งที่ทีมควบคุม
ฐานข้อมูล	ใช้	DR-01–DR-13 ต้องเก็บถาวร · NFR-04 ต้องมี transaction
งานตามเวลา	ไม่ใช้ในรุ่นแรก	ไม่มี FR ที่ต้องเกิดตามเวลาโดยไม่มีคนกด — no-show บันทึกโดย Officer (FR-14)

ทำไมเขียนแบบนี้

- เบราว์เซอร์อยู่ในเครื่องของผู้ใช้ ผู้ใช้เปิดดูและแก้โค้ดฝั่งนี้ได้ — จึงห้ามเป็นที่ตัดสินสิทธิ์หรือกติกา
- NFR เลือก "ชนิด" ของเทคโนโลยี ส่วน "ยี่ห้อ" (React, Express, PostgreSQL) มาจาก stack อ้างอิงของรายวิชา — บันทึกไว้ใน DEC-04

5. NFR กระทบ container ไหน

ไล่ NFR ทีละข้อว่ากระทบ container ไหน — ถ้า NFR ยัง TBD เราไม่ออกแบบเกิน

มาลองทำไปด้วยกัน

1. เปิด SRS หัวข้อ 5 อ่าน NFR ทีละข้อ
2. ถามว่า **ข้อนี้ทำให้ต้องวางอะไรไว้ที่ container ไหน**
3. ถ้า NFR ยังมีค่าเป็น TBD (เช่น NFR-05, NFR-07) ให้เขียนว่ายังไม่ออกแบบ และชี้ OI ที่รออยู่

NFR	ต้องการอะไร	ผลต่อ container	สถานะ
NFR-01	ไม่เก็บ password หรือเลขบัตรประชาชน	ไม่มี container ไหนเก็บรหัสผ่าน · ยืนยันตัวตนผ่าน EXT-01	protocol TBD (DD-01, OI-03)
NFR-02	อ่านและทำได้ตาม role · unit scope · ownership	API Application ตรวจทุก action · Web Application แค่ซ่อนเมนู	DEC-01
NFR-03	audit ทุก state change และ decision	API Application เขียน AuditEvent ลง Database พร้อมการเปลี่ยน state	DEC-01, DEC-02
NFR-04	Approve พร้อมกันต้องไม่เกิด Confirmed ซ้อน	recheck ที่ API Application + Database เดียวที่รองรับ transaction	DEC-02 · กลไกจริง = DD-02
NFR-05	ค้นหาเร็วพอภายใต้ workload ที่ตกลง	ยังไม่เพิ่ม cache หรือ search engine	TBD — OI-01 (DD-04)
NFR-06	flow หลักใช้ keyboard ได้ มี label/status/error ชัด	Web Application — ออกแบบหน้าจอใน W12	ส่งต่อ W12
NFR-07	ป้องกันข้อมูลสูญหายตาม recovery objective	backup/restore เป็นเรื่อง deployment — ไม่ออกแบบใน W11	TBD — OI-02 (DD-05)
NFR-08	log มี correlation reference ไม่เปิด PII เกินจำเป็น	API Application เขียน operational log ตามกติกานี้	retention TBD — OI-02

ทำไมเขียนแบบนี้

- ตารางนี้ทำให้เห็นว่า NFR ไม่ได้ลอยอยู่ใน SRS — ทุกข้อมีที่อยู่ในแบบ หรือถูกบันทึกว่ารอใครตอบ
- ถ้าเราเดาตัวเลขของ NFR ที่ยัง TBD เอง ตัวเลขนั้นจะกลายเป็น requirement ปลอมที่ไม่มีใครอนุมัติ

6. Decision cards

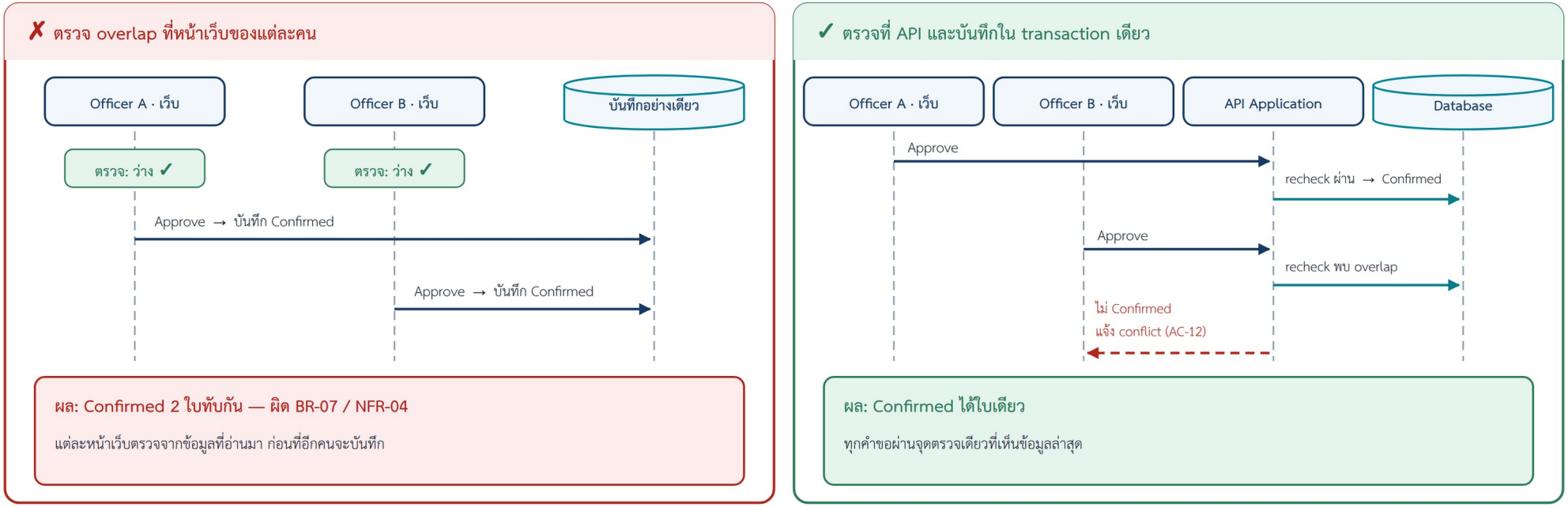
ทุกการตัดสินใจที่ Dev อาจถามว่า "ทำไม" เราเขียนเป็น decision card (ADR ย่อ) — CRBS มี 3 ใบจาก NFR/driver + 1 ใบเรื่อง stack — ใบ stack **บังคับเฉพาะทีมที่เปลี่ยนจาก stack อ้างอิง** CRBS ใช้ stack อ้างอิง แต่เขียนไว้ให้ดูรูปแบบ (ภาพที่ 5 อยู่ถัดจาก DEC-02)

DEC-01 กติกาและการตรวจสิทธิ์อยู่ที่ API Application ไม่ใช่ที่หน้าเว็บ [Proposed]	
ตัดสินใจ	API Application เป็นที่เดียวที่ตรวจกติกา BR-01–BR-10 และสิทธิ์ของทุก action
เพราะ (NFR / driver)	NFR-02 · NFR-03 · NFR-04 · D2 · D3
ทางเลือกที่ไม่เลือก	ตรวจที่หน้าเว็บอย่างเดียว — ผู้ใช้ข้ามหน้าเว็บได้ และแต่ละหน้าเห็นข้อมูลคนละช่วงเวลา (ภาพที่ 5)
ผลที่ตามมา	หน้าเว็บยังตรวจเบื้องต้นได้เพื่อความสะดวก แต่ผลตัดสินจริงอยู่ที่ API

DEC-02 ฐานข้อมูลเดียวที่รองรับ transaction เป็นแหล่งของ Confirmed [Proposed]	
ตัดสินใจ	เก็บ DR-01–DR-13 ใน Database เดียว และให้ recheck กับการบันทึก Confirmed อยู่ใน transaction เดียวกัน
เพราะ (NFR / driver)	NFR-04 · NFR-03 · DR-07/09/10 ผูกกันแน่น · D2
ทางเลือกที่ไม่เลือก	แยกที่เก็บตามหน้าที่ — ต้องประสานหลายที่ ทำให้ D2 ยากขึ้น
ผลที่ตามมา	ชนิดฐานข้อมูลชัดแล้ว (เชิงสัมพันธ์ รองรับ transaction) · กลไกกันพร้อมกันจริง = DD-02

ทำไม "ที่อยู่ของกติกา" สำคัญ — D2: Confirmed ห้ามซ้อน (BR-07, NFR-04)

สถานการณ์: Officer 2 คนกด Approve คำขอ 2 ใบ ของห้องเดียวกัน ช่วงเวลาทับกัน ในเวลาเดียวกัน



W11 ตัดสินว่า "ตรวจที่ไหน" → API Application + Database เดียว · กลไกกันการ Approve พร้อมกันจริง = DD-02 ให้ Dev เลือกและทดสอบ

ภาพที่ 5 ทำไมที่อยู่ของกติกาสำคัญ — D2 ของ CRBS เมื่อ Officer 2 คน Approve พร้อมกัน

DEC-03 เว็บแอปเดียว แสดงหน้าตาม role [Proposed]	
ตัดสินใจ	Requester และ Officer ใช้ Web Application เดียวกัน แต่เห็นเมนูและหน้าตาม role
เพราะ (NFR / driver)	ทุก actor ใช้ผ่านหน้าจอทั่วไป · NFR-06 · SRS ไม่มี requirement ที่ต้องใช้ความสามารถเฉพาะของมือถือ
ทางเลือกที่ไม่เลือก	แอปมือถือ หรือแยกแอปของ Officer — เพิ่มงานโดยไม่มี requirement รองรับ
ผลที่ตามมา	W12 ออกแบบหน้าจอบนเว็บ · สิทธิ์จริงยังตรวจที่ API (DEC-01)

DEC-04 ใช้ stack อ้างอิงของรายวิชา: React · Node.js + Express · PostgreSQL [Proposed]	
ตัดสินใจ	Web Application = React · API Application = Node.js + Express (REST API ส่ง JSON ผ่าน HTTPS) · Database = PostgreSQL
เพราะ (NFR / driver)	ตรงกับชนิดที่ DEC-01–03 ต้องการ · PostgreSQL รองรับ transaction (NFR-04) · หน้าเว็บกับ API ใช้ JavaScript ภาษาเดียว และทีมคุ้นเคย
ทางเลือกที่ไม่เลือก	stack อื่นในเมนู เช่น Python + FastAPI หรือ MySQL — ใช้ได้เหมือนกัน แต่ไม่มีเหตุผลจาก NFR ให้เปลี่ยน
ผลที่ตามมา	version และการตั้งโปรเจกต์ = DD-06 ให้ Dev ยืนยัน · ไม่ใช้ Docker/Kubernetes, microservice หรือ message broker ในรุ่นแรก

ทำไมเขียนแบบนี้

- decision card ตอบคำถาม "ทำไม" ไว้ล่วงหน้า — Dev ที่มาใหม่อ่านแล้วไม่ต้องเดา และไม่เปลี่ยนเองเงียบ ๆ
- ใส่ "ทางเลือกที่ไม่เลือก" ทุกครั้ง เพื่อให้เห็นว่าเราเปรียบเทียบแล้ว ไม่ได้เลือกเพราะชอบ
- สถานะ Proposed = ข้อเสนอของทีมออกแบบ ทีม Dev ยืนยันหรือขอเปลี่ยนได้

7. เดิน Spine บน container diagram

เดินคำขอหนึ่งใบทีละขั้นบนภาพที่ 3 — ทุกขั้นต้องบอกได้ว่าผ่าน container ไหน และกติกาไหนถูกตรวจที่ไหน

#	ขั้นในเส้นทางคำขอหนึ่งใบ	UC	W10 ระดับ	W11 เดินผ่าน container
1	ค้นหาทรัพยากรที่ว่าง	UC-01	Build now	Web → API (นับเฉพาะ Confirmed + block) → Database
2	สร้าง/แก้ Draft (ยังไม่กันทรัพยากร)	UC-02	Build now	Web → API → Database
3	Submit → Submitted + เลขอ้างอิง → precheck	UC-02	Build now	Web → API (precheck · ออกเลขอ้างอิง) → Database

#	ขั้นในเส้นทางคำขอหนึ่งใบ	UC	W10 ระดับ	W11 เดินผ่าน container
4	Officer ตัดสิน → recheck → Confirmed / Rejected / Needs Information	UC-04	Build now	Web (Officer) → API (recheck + เปลี่ยน state + audit ใน transaction เดียว) → Database
5	Requester ดูสถานะคำขอตนเอง	UC-03	Build now (ดูอย่างเดียว)	Web → API (เฉพาะคำขอของตน — NFR-02) → Database
6	Check Out	UC-07	Overview	Web (Officer) → API → Database
7	Return	UC-08	Overview	Web (Officer) → API → Database
ทุกแถว	สิทธิ์ + audit	—	Build now (cross-cutting)	ตรวจสิทธิ์ที่ API ทุก action · AuditEvent ลง Database

ทำไมเขียนแบบนี้

- ทุกขั้นเดินเส้นเดียวกัน Web → API → Database — แปลว่าโครงสร้างรุ่นแรกเรียบพอ ความซับซ้อนอยู่ข้างใน API ซึ่ง W13 จะซูมเข้าไป (C4 ระดับ 3)
- อัปเดตไฟล์ Spine ของทีมด้วย — คอลัมน์ "W11 กล่องที่รับผิดชอบ" ในไฟล์ = คอลัมน์ "W11 เดินผ่าน container" ในตารางนี้ (คอลัมน์ W12–W13 ยังว่างไว้)

8. Open decisions ที่กระทบ container

ID	เรื่อง	กระทบ container	ระหว่างรอทำอย่างไร	เจ้าของคำตอบ
DD-01	identity service และ protocol/field ที่ส่งมา (OI-03)	API Application + EXT-01	role fixture: ผู้ใช้ทดสอบพร้อม role ที่กำหนดล่วงหน้า · ไม่เก็บรหัสผ่าน	Security/Interface reviewer (ผู้สอนในกรณีสอน)
DD-02	กลไกกันสอง Approve พร้อมกัน	API Application + Database	recheck ก่อน Confirmed ทุกครั้ง · Dev เลือกกลไกและเขียน race scenario ไว้ทดสอบ	Dev
DD-03	precheck พบ block ในรุ่นแรก	API Application + Web Application	Officer เห็น "รอ Area Manager" อ่านอย่างเดียว	ผู้สอน / Product owner ของกรณีสอน
DD-04	เป้าหมาย performance ของการค้นหา (OI-01)	API Application + Database	ไม่กำหนดตัวเลข · ยังไม่เพิ่ม cache	Product Owner / Instructor

ID	เรื่อง	กระทบ container	ระหว่างรอทำอย่างไร	เจ้าของคำตอบ
DD-05	retention / backup / recovery (OI-02)	Database	ไม่ลบ audit/decision ในรุ่นแรก · backup เป็นงาน deployment	Data/Operations reviewer
DD-06 (ใหม่)	ยืนยัน stack และ version	ทุก container	ใช้ stack อ้างอิงตาม DEC-04 ระหว่างรอ	Dev

ทำไมเขียนแบบนี้

- ภาพต้องบอกได้ว่าเรื่องที่ยังไม่ตัดสินไปตกที่ container ไหน — Dev จะรู้ว่าส่วนไหนเริ่มได้เลย ส่วนไหนต้องใช้วิธีชั่วคราว
- stack ในแบบเป็นข้อเสนอ (Proposed) จึงต้องมี DD-06 ให้ Dev ยืนยัน — ไม่ใช่ข้อกำหนดตายตัว

9. ส่งต่อ W12 และ W13

- **W12:** ทุกหน้าจอ (ค้นหา · ฟอร์มคำขอ · คำขอของฉัน · หน้าตัดสิน) อยู่ใน Web Application และระบุว่าเรียก API เรื่องอะไร
- **W12:** ข้อความบนหน้าจอตรงกับ Request State (Submitted = ส่งแล้ว รอพิจารณา ยังไม่กันทรัพยากร) · NFR-06 ออกแบบในหน้าจอ
- **W13:** ซูมเข้า API Application เป็น component diagram (C4 ระดับ 3) พร้อมเจ้าของข้อมูล
- **W13:** sequence ใช้ lifeline = Web Application + component ใน API + Database · action card อ้าง DEC-01 และ DEC-02
- **W14:** Container diagram + decision cards = หน้าแรกของ Developer Handoff ที่ทีมนำเสนอและส่งมอบให้ Dev

10. Done checklist W11

- ☑ Context diagram: ระบบ 1 กล่อง + actor ครบ + ระบบภายนอกพร้อมป้าย Core / Extension / TBD
- ☑ Container diagram: ทุก container มีชื่อ + [ชนิด · เทคโนโลยี] + หน้าที่; ลูกศรทุกเส้นบอกว่าส่งอะไร
- ☑ ตาราง container บอกสภาพแวดล้อม (รันที่ไหน ใครควบคุม) และข้อมูลที่เก็บ
- ☑ NFR ทุกข้อชี้ได้ว่ากระทบ container ไหน หรือเป็น TBD; decision card ≤ 3 ใบอ้าง NFR/driver
- ☑ ทุกขั้น Build now/Overview ใน Spine เดินผ่าน container ได้ และกติกา D1–D3 อยู่ฝั่ง server
- ☑ ใช้ stack อ้างอิง หรือมี decision card เมื่อเปลี่ยน; ไม่มีเทคโนโลยีเกินจำเป็น; ทุกภาพมี .drawio + PNG

11. AI use และ Team worklog

เครื่องมือ	ใช้ช่วยเรื่องใด	ตรวจอย่างไร
AI (Claude)	จัดโครงเอกสารและร่างภาพ	ผู้สอนตรวจกับ SRS W07, Actor and System Context, Domain and Data Scope และ Design Strategy W10 — ไม่เพิ่ม requirement หรือค่าที่แต่งขึ้น

สมาชิก	บทบาท	งานที่ทำ (ระบุหัวข้อ/ภาพ)	หลักฐาน/ลิงก์
ผู้สอน (ตัวอย่าง)	ผู้จัดทำ	หัวข้อ 1–11, ภาพที่ 1–5	ไฟล์ .drawio ของภาพที่ 1–5